

Projeto para criar aplicações clientes do servidor //\ar\carai

Objetivo.

- Desenvolver aplicativos clientes http para as seguintes estruturas de framework:
 - FreePascal - LCL
 - Implementado.
 - JavaScript.
 - Implementado parcialmente.
 - HTML dinâmico.
 - Implementado.
 - Vuejs.
 - Não implementado.
 - Angularjs.
 - Não implementado.
 - Reactjs.
 - Não implementado.

DataModules.

- TCreateClientes
 - Objetivo.
 - Contêm os parâmetros necessários para criar as aplicações clientes, bem como a definição da tela e do banco de dados usados para salvar os parâmetros para cada aplicação configurada.
 - Eventos:
 - OnAddTemplate.
 - DmxScroller_Form1AddTemplate.
 - Objetivo.
 - Definição dos recursos usados para desenhar os formulários bem como mapeamento dos campos do banco de dados.
 - Nota:
 - Os dados deste template serão armazenados no arquivo com nome do DataModule com a extensão .Json_.
 - Campos.
 - Id = Número sequencial gerado automaticamente.
 - Numero sequencial do registro.

- typApp = Tipo de aplicação.
 - App LCL : sim.
 - App Javascript: não.
 - App Dynamic_html:sim.
 - App VueJS: não.
 - App AngularJS: não.
 - App ReactJS: não.
- Nome da pasta Root das aplicações clientes.
 - Obs.
 - O padrão é a pasta do executável da aplicação servidora.
- Nome da Subpasta da aplicação cliente usada para gerar a aplicação.
 - Obs.:
 - Este unit deve estar na aplicação servidora, onde as subpastas são relativas à pasta root usada pelo servidor.
- **Exemplo de árvore gerada:**

```

.
├── lcl
│   └── templates
│       ├── mi_rtl_web_module.dfm
│       ├── mi_rtl_web_module.pas
│       └──
mi_rtl_web_module.form.dfm
├──
mi_rtl_web_module.form.pas
├── dynamic_html
│   ├── css
│   │   ├── color_tons_amarelo.css
│   │   ├── color_tons_dark.css
│   │   ├── color_tons_de_azul_ceu.css
│   │   ├── color_tons_de_cinza.css
│   │   └──
color_tons_de_dark_claro.css
│   ├──
color_tons_de_lilas_claro.css
│   ├──
color_tons_de_verde_claro.css
│   │   ├── MiEditForm.css
│   │   └── MiUiDmxScroller.css
│   └── js
│       ├── MiMessageBox.js
│       ├── MiThemeDialog.js
│       ├── MiConsts.js
│       └── MiMethods.js
│           └── MiUiDmxScroller.js
MiUiDmxScrollerForm.js
├──
MiUiDmxScrollerForm.MiAbstractDialogs.js
├──
MiUiDmxScrollerForm.MiDialogs.js
│   ├──
│   ├──
│   └──
│       ├── MiDates.js
│       ├── MiMaskEdit.js
│       └── MiEditForm.js
│   └── templates
│       └── MiEditForm.html
└── Tmi_rtl_web_module.form.html

```

- Nome do arquivo destino do código gerado.
 - O nome do arquivo destino será o mesmo nome do WebModule server com a extensão compatível com a aplicação destino.
 - Exemplo:
 - Modulo servidor:
 - Classe **Tmi_rtl_web_module** é o

modulo servidor. - Arquivo cliente a ser gerado para o módulo servidor

Tmi_rtl_web_module são:

- **Tmi_rtl_web_module.pas** se app

FreePascal - LCL. -

Tmi_rtl_web_module.js se app

javascript. -

Tmi_rtl_web_module.html se app

HTML dinâmico.

- **Tmi_rtl_web_module.vue** se App

VueJS.

- **Tmi_rtl_web_module** App AngularJS.

- Tmi_rtl_web_module.module.ts:

Configuração do módulo principal da aplicação. -

Tmi_rtl_web_module.component.ts:

Componente raiz da aplicação. -

Tmi_rtl_web_module.component.html:

Template do componente raiz. -

Tmi_rtl_web_module.component.js: O

transpilador angular gera o javascript a partir do typescript -

Tmi_rtl_web_module.service.ts:

Serviço para fazer chamadas HTTP

para a API RESTful. -

Tmi_rtl_web_module.jsx App ReactJS.

■ **Notas:**

- A versão de lançamento do projeto marcará e vai gerar aplicações clientes somente para LCL e HTML dinâmico.

■

■